



Girls' Programming Network

Candy Catcher!

FOR TUTOR EYES ONLY

This project was created by GPN Australia for GPN sites all around Australia!

This workbook and related materials were created by tutors at:

Sydney and Melbourne



Girls' Programming Network

If you see any of the following tutors don't forget to thank them!!

Writers

Isabella Hogan

Angela Sojan

Sophie Lee-Goh

Testers

Amanda Hogan

Scaffold

```
# <insert your name here>
# imports
import random
import pgzrun

# create constants
WIDTH = 100
HEIGHT = 100
SQUARE = 32
NUM_SQUARES_WIDTH = WIDTH // SQUARE
NUM_SQUARES_HEIGHT = HEIGHT // SQUARE

# print welcome

# make hero

# make candy

# add candy

# add game bits

# make timer (extension)

# start candy positions

# manage key controls
def on_key_down(key):
    # down up left right

    # stay on screen (extension)
    pass

# update everything
def update():
    # check collisions

    # win condition
```

```
    # timer (extension)
    pass

# draw everything to screen
def draw():

    # empty screen

    # draw candies

    # draw hero

    # draw text on screen

# runs everything
pgzrun.go()
```

Part 1: Welcome to “Candy Catcher”

1.1: Who am I?

The code should look like this (no bonuses):

```
...  
# My name is Angela  
# imports  
...
```

1.2: Welcome to ‘Candy Catcher’

The code should look like this (no bonuses):

```
...  
# print welcome  
print("""-----  
Welcome to Candy Catcher!  
The game is about to start.  
Press the arrow keys to move  
and collect the candies.  
-----""")  
...
```

1.3: Fix the screen

The code should look like this (no bonuses):

```
...  
# create constants  
WIDTH = 640  
HEIGHT = 480  
...
```

Full solution after Part 1

The code should look like this (no bonuses):

```
# My name is Angela  
# imports  
import pgzrun  
  
# create constants  
WIDTH = 640
```

```
HEIGHT = 480
```

```
# print welcome
```

```
print("""-----  
Welcome to Candy Catcher!  
The game is about to start.  
Press the arrow keys to move  
and collect the candies.  
-----""")
```

```
# make hero
```

```
# make candy
```

```
# add candy
```

```
# add game bits
```

```
# make timer (extension)
```

```
# start candy positions
```

```
# manage key controls
```

```
    # down up left right
```

```
    # stay on screen (extension)
```

```
# update everything
```

```
    # check collisions
```

```
    # win condition
```

```
    # timer (extension)
```

```
# draw everything to screen
```

```
def draw():
```

```
# empty screen

# draw candies

# draw hero

# draw text on screen

# runs everything
pgzrun.go()
```

Part 2: Holding out for a hero

2.1: Creating the hero

The code should look like this (no bonuses):

```
...
# make hero
hero = Actor("hero")
...
```

2.2: Drawing the hero to the screen

The code should look like this (no bonuses):

```
...

# draw everything to screen
def draw():
    # empty screen

    # draw candies

    # draw hero
    hero.draw()
...
```

Note: students have been asked to remove “pass” from the draw function. “pass” is required in the scaffold, as Python does not allow empty function declarations. If “pass” is left in the code, however, that is fine and will not impact the code functionality.

Full solution after Part 2

The code should look like this (no bonuses):

```
...
# My name is Angela
# imports
import pgzrun

# create constants
WIDTH = 640
```

```

HEIGHT = 480

# print welcome
print("""-----
Welcome to Candy Catcher!
The game is about to start.
Press the arrow keys to move
and collect the candies.
-----""")

# make hero
hero = Actor("hero")

# make candy

# add candy

# add game bits

# make timer (extension)

# start candy positions

# manage key controls

    # down up left right

    # stay on screen (extension)

# update everything

    # check collisions

    # win condition

    # timer (extension)

# draw everything to screen
def draw():

```

```
# empty screen

# draw candies

# draw hero
hero.draw()

# draw text on screen

# runs everything
pgzrun.go()
...
```

Part 3: I like to move it, move it!

3.1: Adding the maths

The code should look like this (no bonuses):

```
...
# create constants
WIDTH = 640
HEIGHT = 480
SQUARE = 32
...
```

3.2: Moving the hero down

The code should look like this (no bonuses):

```
...
# manage key controls
def on_key_down(key):

    # down up left right
    hero.bottom = hero.bottom + SQUARE
...
```

3.3: Moving down when we want to

The code should look like this (no bonuses):

```
...
# manage key controls
def on_key_down(key):

    # down up left right
    if key == keys.DOWN:
        hero.bottom = hero.bottom + SQUARE
...
```

3.4: Adding other moves

The code should look like this (no bonuses):

```
...
# manage key controls
def on_key_down(key):

    # down up left right
    if key == keys.DOWN:
        hero.bottom = hero.bottom + SQUARE
    if key == keys.UP:
        hero.top = hero.top - SQUARE
    if key == keys.LEFT:
        hero.left = hero.left - SQUARE
    if key == keys.RIGHT:
        hero.right = hero.right + SQUARE
...
```

3.5: Painting it black

The code should look like this (no bonuses):

```
...
def draw():

    # empty screen
    screen.fill("black")
...
```

Full solution after Part 3

The code should look like this (no bonuses):

```
...
# My name is Angela
# imports
import pgzrun

# create constants
WIDTH = 640
HEIGHT = 480
SQUARE = 32
```

```

# print welcome
print("""-----
Welcome to Candy Catcher!
The game is about to start.
Press the arrow keys to move
and collect the candies.
-----""")

# make hero
hero = Actor("hero")

# make candy

# add candy

# add game bits

# make timer (extension)

# start candy positions

# manage key controls
def on_key_down(key):

    # down up left right
    if key == keys.DOWN:
        hero.bottom = hero.bottom + SQUARE
    if key == keys.UP:
        hero.top = hero.top - SQUARE
    if key == keys.LEFT:
        hero.left = hero.left - SQUARE
    if key == keys.RIGHT:
        hero.right = hero.right + SQUARE

    # stay on screen (extension)

# update everything

    # check collisions

    # win condition

```

```

    # timer (extension)

# draw everything to screen
def draw():

    # empty screen
    screen.fill("black")

    # draw candies

    # draw hero
    hero.draw()

    # draw text on screen

# runs everything
pgzrun.go()
...

```

Bonus 3.6: Keeping the hero on screen

```

...
# stay on screen (extension)
    if hero.top < 0:
        hero.top = 0
    if hero.bottom > HEIGHT:
        hero.bottom = HEIGHT
    if hero.left < 0:
        hero.left = 0
    if hero.right > WIDTH:
        hero.right = WIDTH
...

```

Part 4: I want candy!

4.1: Creating a candy

The code should look like this (no bonuses):

```
...  
# make candy  
candy = Actor("candy")  
  
...
```

4.2: Putting it on the screen in a random position

The code should look like this (no bonuses):

```
...  
# imports  
import pgzrun  
import random  
  
# start candy positions  
candy.bottom = random.randint(SQUARE, HEIGHT)  
candy.right = random.randint(SQUARE, WIDTH)  
  
# draw candies  
    candy.draw()  
  
...
```

4.3: Adding more candies

The code should look like this (no bonuses):

```
...
# add candy
candy2 = Actor("candy")
candy3 = Actor("candy")

# start candy positions
candy.bottom = random.randint(SQUARE, HEIGHT)
candy.right = random.randint(SQUARE, WIDTH)
candy2.bottom = random.randint(SQUARE, HEIGHT)
candy2.right = random.randint(SQUARE, WIDTH)
candy3.bottom = random.randint(SQUARE, HEIGHT)
candy3.right = random.randint(SQUARE, WIDTH)

# draw candies
    candy.draw()
    candy2.draw()
    candy3.draw()
...
```

4.4: Collecting candies

The code should look like this (no bonuses):

```
...
# update everything
def update():

    # check collisions
    if hero.colliderect(candy):
        print("Collided!")
...
```

4.5: Moving the candies

The code should look like this (no bonuses):

```
...
# update everything
def update():

    # check collisions
    if hero.colliderect(candy):
        candy.bottom = random.randint(SQUARE, HEIGHT)
        candy.right = random.randint(SQUARE, WIDTH)
    if hero.colliderect(candy2):
        candy2.bottom = random.randint(SQUARE, HEIGHT)
        candy2.right = random.randint(SQUARE, WIDTH)
    if hero.colliderect(candy3):
        candy3.bottom = random.randint(SQUARE, HEIGHT)
        candy3.right = random.randint(SQUARE, WIDTH)

...
```

Full solution after Part 4

The code should look like this (no bonuses):

```
# My name is Angela
# imports
import pgzrun
import random

# create constants
WIDTH = 640
HEIGHT = 480
SQUARE = 32

# print welcome
print("""-----
Welcome to Candy Catcher!
The game is about to start.
Press the arrow keys to move
and collect the candies.
-----""")

# make hero
hero = Actor("hero")
```

```

# make candy
candy = Actor("candy")

# add candy
candy2 = Actor("candy")
candy3 = Actor("candy")

# add game bits

# make timer (extension)

# start candy positions
candy.bottom = random.randint(SQUARE, HEIGHT)
candy.right = random.randint(SQUARE, WIDTH)
candy2.bottom = random.randint(SQUARE, HEIGHT)
candy2.right = random.randint(SQUARE, WIDTH)
candy3.bottom = random.randint(SQUARE, HEIGHT)
candy3.right = random.randint(SQUARE, WIDTH)

# manage key controls
def on_key_down(key):

    # down up left right
    if key == keys.DOWN:
        hero.bottom = hero.bottom + SQUARE
    if key == keys.UP:
        hero.top = hero.top - SQUARE
    if key == keys.LEFT:
        hero.left = hero.left - SQUARE
    if key == keys.RIGHT:
        hero.right = hero.right + SQUARE

    # stay on screen (extension)

# update everything
def update():

    # check collisions
    if hero.colliderect(candy):
        candy.bottom = random.randint(SQUARE, HEIGHT)
        candy.right = random.randint(SQUARE, WIDTH)
    if hero.colliderect(candy2):
        candy2.bottom = random.randint(SQUARE, HEIGHT)

```

```
        candy2.right = random.randint(SQUARE, WIDTH)
    if hero.colliderect(candy3):
        candy3.bottom = random.randint(SQUARE, HEIGHT)
        candy3.right = random.randint(SQUARE, WIDTH)

    # win condition

    # timer (extension)

# draw everything to screen
def draw():

    # empty screen
    screen.fill("black")

    # draw candies
    candy.draw()
    candy2.draw()
    candy3.draw()

    # draw hero
    hero.draw()

    # draw text on screen

# runs everything
pgzrun.go()
```

Part 5: A better game!

5.1: Keeping count

The code should look like this (no bonuses):

```
...
# add game bits
score = 0

# update everything
def update():

    global score

    # check collisions
    if hero.colliderect(candy):
        candy.top = random.randint(0, NUM_SQUARES_HEIGHT - 1) * SQUARE
        candy.left = random.randint(0, NUM_SQUARES_WIDTH - 1) * SQUARE
        score = score + 1
    if hero.colliderect(candy2):
        candy2.top = random.randint(0, NUM_SQUARES_HEIGHT - 1) * SQUARE
        candy2.left = random.randint(0, NUM_SQUARES_WIDTH - 1) * SQUARE
        score = score + 1
    if hero.colliderect(candy3):
        candy3.top = random.randint(0, NUM_SQUARES_HEIGHT - 1) * SQUARE
        candy3.left = random.randint(0, NUM_SQUARES_WIDTH - 1) * SQUARE
        score = score + 1
...
```

5.2: Showing the score

The code should look like this (no bonuses):

```
...
# draw text on screen
x_pos = 300
y_pos = 10
screen.draw.text('Score: ' + str(score), (x_pos, y_pos),
color=(255,255,255), fontsize=30)
...
```

Full solution after Part 5

The code should look like this (no bonuses):

```
...
# My name is Angela
# imports
import pgzrun
import random

# create constants
WIDTH = 640
HEIGHT = 480
SQUARE = 32

# print welcome
print("""-----
Welcome to Candy Catcher!
The game is about to start.
Press the arrow keys to move
and collect the candies.
-----""")

# make hero
hero = Actor("hero")

# make candy
candy = Actor("candy")

# add candy
candy2 = Actor("candy")
candy3 = Actor("candy")

# add game bits
score = 0

# make timer (extension)

# start candy positions
candy.bottom = random.randint(SQUARE, HEIGHT)
candy.right = random.randint(SQUARE, WIDTH)
candy2.bottom = random.randint(SQUARE, HEIGHT)
candy2.right = random.randint(SQUARE, WIDTH)
```

```

candy3.bottom = random.randint(SQUARE, HEIGHT)
candy3.right = random.randint(SQUARE, WIDTH)

# manage key controls
def on_key_down(key):

    # down up left right
    if key == keys.DOWN:
        hero.bottom = hero.bottom + SQUARE
    if key == keys.UP:
        hero.top = hero.top - SQUARE
    if key == keys.LEFT:
        hero.left = hero.left - SQUARE
    if key == keys.RIGHT:
        hero.right = hero.right + SQUARE

    # stay on screen (extension)

# update everything
def update():

    global score

    # check collisions
    if hero.colliderect(candy):
        candy.bottom = random.randint(SQUARE, HEIGHT)
        candy.right = random.randint(SQUARE, WIDTH)
        score = score + 1
    if hero.colliderect(candy2):
        candy2.bottom = random.randint(SQUARE, HEIGHT)
        candy2.right = random.randint(SQUARE, WIDTH)
        score = score + 1
    if hero.colliderect(candy3):
        candy3.bottom = random.randint(SQUARE, HEIGHT)
        candy3.right = random.randint(SQUARE, WIDTH)
        score = score + 1

    # win condition

    # timer (extension)

# draw everything to screen
def draw():

```

```
# empty screen
screen.fill("black")

# draw candies
candy.draw()
candy2.draw()
candy3.draw()

# draw hero
hero.draw()

# draw text on screen
x_pos = 300
y_pos = 10
screen.draw.text('Score: ' + str(score), (x_pos, y_pos),
color=(255,255,255), fontsize=30)

# runs everything
pgzrun.go()
...
```

Bonus 5.3: Setting a target to win

The code should look like this (no bonuses):

```
...
# add game bits
score = 0
target = 20
win = False

# update everything
def update():

    global score, win

# win condition
    if score == target:
        win = True

# draw everything to screen
def draw():
    if not win:
        # empty screen
        screen.fill("black")
        # draw candies
        candy.draw()
        candy2.draw()
        candy3.draw()
        # draw hero
        hero.draw()
        # draw text on screen
        x_pos = 300
        y_pos = 10
        screen.draw.text('Score: ' + str(score), (x_pos, y_pos),
color=(255,255,255), fontsize=30)
    else:
        screen.draw.text('You win!!', (WIDTH//2, HEIGHT//2), color=(255,255,255),
fontsize=60)

...
```

Part 6: Extension 1- Adding a timer

6.1: Set up variables

The code should look like this (no bonuses):

```
...
# make timer (extension)
timer = 30
...
```

6.2: Timer maths

The code should look like this (no bonuses):

```
...
# update everything
def update():

    global score, win, timer

# timer (extension)
    timer = timer - 1/60
...
```

6.3: Draw timer on screen

The code should look like this (no bonuses):

```
...
# update everything
screen.draw.text('Time: ' + format(timer, ".0f"), (x_pos + 100,
y_pos), color=(255,255,255), fontsize=30)
...
```

Part 7: Extension 2- Game Over!

7.1: Lose yourself

The code should look like this (no bonuses):

```
...
# add game bits
score = 0
target = 20
win = False
lose = False
...
```

7.2: If you run out of time

The code should look like this (no bonuses):

```
...
# timer (extension)
    timer = timer - 1/60
    if timer <= 0:
        lose = True

# update everything
def update():

    global score, win, timer, lose
    if win or lose:
        return

...
```

7.3: Drawing Game Over

The code should look like this (no bonuses):

```
...
if lose:
    screen.draw.text('You lose!!', center=(WIDTH//2, HEIGHT//2),
color=(255,255,255), fontsize=60)
if win:
    screen.draw.text('You win!!', (WIDTH//2, HEIGHT//2),
color=(255,255,255), fontsize=60)

# Remember to remove the if not win condition at the top of the
draw function or to make sure the student gets the right structure
of if/else tests
...
```

Part 8: Extension 3- Changing the background!

8.1: Changing the background

The code should look like this (no bonuses):

```
...  
# empty screen  
    screen.fill("green")  
...
```

Full solution after Extensions

The code should look like this:

```
...  
# My name is Angela  
# imports  
import pgzrun  
import random  
  
# create constants  
WIDTH = 640  
HEIGHT = 480  
SQUARE = 32  
NUM_SQUARES_WIDTH = WIDTH // SQUARE  
NUM_SQUARES_HEIGHT = HEIGHT // SQUARE  
  
# print welcome  
print("""-----  
Welcome to Candy Catcher!  
The game is about to start.  
Press the arrow keys to move  
and collect the candies.  
-----""")  
  
# make hero  
hero = Actor("hero")  
  
# make candy  
candy = Actor("candy")  
  
# add candy
```

```

candy2 = Actor("candy")
candy3 = Actor("candy")

# add game bits
score = 0
target = 20
win = False
lose = False

# make timer (extension)
timer = 30

# start candy positions
candy.top = random.randint(0, NUM_SQUARES_HEIGHT - 1) * SQUARE
candy.left = random.randint(0, NUM_SQUARES_WIDTH - 1) * SQUARE
candy2.top = random.randint(0, NUM_SQUARES_HEIGHT - 1) * SQUARE
candy2.left = random.randint(0, NUM_SQUARES_WIDTH - 1) * SQUARE
candy3.top = random.randint(0, NUM_SQUARES_HEIGHT - 1) * SQUARE
candy3.left = random.randint(0, NUM_SQUARES_WIDTH - 1) * SQUARE

# manage key controls
def on_key_down(key):

    # down up left right
    if key == keys.DOWN:
        hero.bottom = hero.bottom + SQUARE
    if key == keys.UP:
        hero.top = hero.top - SQUARE
    if key == keys.LEFT:
        hero.left = hero.left - SQUARE
    if key == keys.RIGHT:
        hero.right = hero.right + SQUARE

    # stay on screen (extension)

# update everything
def update():

    global score, win, timer, lose
    if win or lose:
        return
    # check collisions
    if hero.colliderect(candy):
        candy.top = random.randint(0, NUM_SQUARES_HEIGHT - 1) * SQUARE
        candy.left = random.randint(0, NUM_SQUARES_WIDTH - 1) * SQUARE

```

```

        score = score + 1
    if hero.colliderect(candy2):
        candy2.top = random.randint(0, NUM_SQUARES_HEIGHT - 1) * SQUARE
        candy2.left = random.randint(0, NUM_SQUARES_WIDTH - 1) * SQUARE
        score = score + 1
    if hero.colliderect(candy3):
        candy3.top = random.randint(0, NUM_SQUARES_HEIGHT - 1) * SQUARE
        candy3.left = random.randint(0, NUM_SQUARES_WIDTH - 1) * SQUARE
        score = score + 1

# win condition
if score == target:
    win = True

# timer (extension)
timer = timer - 1/60
    if timer <= 0:
        lose = True

# draw everything to screen
def draw():

    # empty screen
    screen.fill("green")

    # draw candies
    candy.draw()
    candy2.draw()
    candy3.draw()

    # draw hero
    hero.draw()

    # draw text on screen
    x_pos = 300
    y_pos = 10
    screen.draw.text('Score: ' + str(score), (x_pos, y_pos),
color=(255,255,255), fontsize=30)
    screen.draw.text('Time: ' + format(timer, ".0f"), (x_pos +
100, y_pos), color=(255,255,255), fontsize=30)
    if lose:
        screen.draw.text('You lose!!', center=(WIDTH//2,
HEIGHT//2), color=(255,255,255), fontsize=60)
    if win:

```

```
        screen.draw.text('You win!!', (WIDTH//2, HEIGHT//2),
color=(255,255,255), fontsize=60)
```

```
# runs everything
```

```
pgzrun.go()
```

```
...
```